Describe the data structure you used to implement the graph and why. [2.5 points]

In the Project 2 breakdown video it was recommended that we use hashmaps or maps in our implementation. Since I am more comfortable using maps that is the data structure I choose. To store the page rank values I used a map rather than an unordered_map to keep the elements in a sorted order based on their keys. This also allowed me to sort the values alphabetically. For the in and out degrees I used two unordered maps. I choose to use unordered maps for this part of my implementation. I did this as in an unordered map searching for an element or inserting into the unordered map has an average O(1) time complexity. Making it more efficient for storing and accessing the in degrees and out degrees of the webpages, which is why I went with the unordered map rather than the regular sorted map.

What is the computational complexity of each method in your implementation in the worst case in terms of Big O notation? [5 points]

void initialRank();

The worst case time complexity for this function would be O(v) where v represents the number of vertices in the graph. This is because we have a loop that runs until it reaches the end of inDeg, which means the number of iterations relies on how many elements (vertices) are stored in inDeg. Inside the loop we are adding elements into an unordered map which would give O(1) time complexity. Making the time complexity for the whole function O(v).

int numVertices();

This functions time complexity in the worst case is just O(1). This because the function only contains one line, no loops, and uses .size() which has a time complexity of O(1).

void insertEdge(string from, string to);

The time complexity in the worst case would be O(1). This is this function mainly is just adding elements into unsorted maps which is O(1) time complexity.

string PageRank(int n);

The worst case time complexity of this function would be O(n * v * L). This is as there are 3 loops in the function. The outer loop runs O(n) times where n represents the number of power iterations inputted by the user. The inner loop has a time complexity of O(v). This is because it is iterating over each vertex in inDeg similarly to the void initialRank(); function. Then the innermost loop has a time complexity of O(L) this is because this loop iterates over the amount of links or edges to each webpage. This results in the time complexity for this function to be O(n * v * L).

What is the computational complexity of your main method in your implementation in the worst case in terms of Big O notation? [5 points]

The computational time complexity of my main method would be O(n * v * L) + O (j) this would be as in my main function there is a loop that iterates depending on the number of lines the user has inputted. The number of lines is represented by j, which makes the time complexity of the loop O(j). After the loop exits the PageRank function is called which we have established above as having a time complexity of O(n * v * L). This is as there are 3 loops in the function. Where the outer loop time complexity is O(n) where n is the number of power iterations. The innerloop time complexity is O(v) where v is the number of vertices looping over inDeg. With the innermost loop being O(L) where L is the average number of links or edges of each webpage. This results in the time complexity of my main method being O(n * v * L) + O (j) .The O(j) portion is added to O(n * v * L) since my PageRank function is called outside of the loop.

What did you learn from this assignment, and what would you do differently if you had to start over? [2 points]

From this assignment I because more familiar with implementing graphs. If I had to start earlier I would read up more on how to write test cases since I did not have time to complete them for project 1 they were a little confusing for me to write.